

Node Role-Guided LLMs for Dynamic Graph Clustering

Dongyuan Li
The University of Tokyo
Tokyo, Japan
lidy@csis.u-tokyo.ac.jp

Ying Zhang
RIKEN
Sendai, Japan
ying.zhang@riken.jp

Yaozu Wu
The University of Tokyo
Tokyo, Japan
yaozuwu279@gmail.com

Renhe Jiang*
The University of Tokyo
Tokyo, Japan
jiangrh@csis.u-tokyo.ac.jp

Abstract

Dynamic graph clustering aims to detect and track time-varying clusters in dynamic graphs, revealing how complex real-world systems evolve over time. However, existing methods are predominantly black-box models. They lack interpretability in their clustering decisions and fail to provide semantic explanations of why clusters form or how they evolve, severely limiting their use in safety-critical domains such as healthcare or transportation. To address these limitations, we propose an end-to-end interpretable framework that maps continuous graph embeddings into discrete semantic concepts through learnable prototypes. Specifically, we first decompose node representations into orthogonal role and clustering subspaces, so that nodes with similar roles (e.g., hubs, bridges) but different cluster affiliations can be properly distinguished. We then introduce five node role prototypes (Leader, Contributor, Wanderer, Connector, Newcomer) in the role subspace as semantic anchors, transforming continuous embeddings into discrete concepts to facilitate LLM understanding of node roles within communities. Finally, we design a hierarchical LLM reasoning mechanism to generate both clustering results and natural language explanations, while providing consistency feedback as weak supervision to refine node representations. Experimental results on four synthetic and six real-world benchmarks demonstrate the effectiveness, interpretability, and robustness of DyG-RoLLM. Code is available at <https://github.com/Clearloveyuan/DyG-RoLLM>.

CCS Concepts

• Computing methodologies → Temporal reasoning.

Keywords

Graph Clustering, Dynamic Graphs, Temporal Networks, Community Detection, LLM Reasoning, Interpretable Graph Learning.

ACM Reference Format:

Dongyuan Li, Ying Zhang, Yaozu Wu, and Renhe Jiang. 2026. Node Role-Guided LLMs for Dynamic Graph Clustering. In *Proceedings of the ACM Web Conference 2026 (WWW '26)*, April 13–17, 2026, Dubai, United Arab Emirates. ACM, New York, NY, USA, 11 pages. <https://doi.org/10.1145/3774904.3792704>

*Corresponding author.



This work is licensed under a Creative Commons Attribution 4.0 International License. *WWW '26, Dubai, United Arab Emirates*
© 2026 Copyright held by the owner/author(s).
ACM ISBN 979-8-4007-2307-0/2026/04
<https://doi.org/10.1145/3774904.3792704>

1 Introduction

Dynamic graph clustering, also known as dynamic community detection, aims to identify and track evolving communities by leveraging both topological structures and temporal dependencies in dynamic graphs [5, 38, 53]. As an effective tool to reveal the evolutionary mechanisms underlying complex real-world systems, dynamic graph clustering has drawn significant attention in various domains, such as social networks analysis [23, 34, 86, 94], recommendation systems [59, 67, 84, 87], and AI4Science [24, 58, 60].

Numerous methods have been proposed for dynamic graph clustering in recent years, which can be broadly classified into three categories [56]. **(i) Neural network-based methods** [15, 39, 74, 76] focus on learning dynamic node representations, with clustering applied as post-processing. This two-stage pipeline separates representation learning from clustering, leading to sub-optimal performance [13, 30]. **(ii) Matrix Factorization-based methods** [4, 12, 30, 36, 37, 43] address this limitation by jointly optimizing representation learning and clustering. These methods cluster nodes at each timestamp while maintaining temporal smoothness, achieving superior performance on multiple benchmarks [28, 50]. Recently, **(iii) LLM-based methods** [17, 49] have emerged to enable semantic reasoning about community structures. These approaches integrate GNNs with LLMs through two paradigms: GNN-as-predictor methods [62, 85] enhance community detection by enriching node embeddings with LLM-extracted semantic features, while LLM-as-predictor approaches [49, 95] directly infer cluster assignments by transforming graph topology into natural language prompts. By incorporating language models, these methods identify clusters based on semantic similarity rather than purely structural connections, enabling the detection of functionally coherent clusters that traditional topology-based clustering might miss [17].

Despite their success, existing dynamic graph clustering methods share a fundamental limitation: **they produce black-box clustering decisions without explanations**, leaving users unable to understand why certain nodes form clusters or how these clusters evolve over time. This lack of interpretability critically limits their adoption in high-risk applications. For instance, in epidemic control, public health officials need to understand not just which clusters are at risk, but why certain groups form transmission clusters and how intervention strategies might affect cluster evolution [65]. Although some works use motifs [6] or causal theory [89] for explanations, such non-textual explanations are abstract, hard to verify, and often fail in complex cases [42]. The emergence of LLMs with strong reasoning capabilities [10] offers a promising direction: using human-understandable natural language to explain clustering decisions. However, enabling faithful explanations for graph clustering raises three challenges: **(i) Clustering-aware Representation**: How can we learn node representations that capture community structure

and temporal dynamics while remaining interpretable for downstream explanations? **(ii) Structure-Semantic Alignment:** How can we present continuous node representations to LLMs so that structural and temporal properties are preserved for language-based reasoning? **(iii) Reasoning Guidance:** How can we guide LLMs to generate consistent, verifiable explanations aligned with the graph rather than hallucinating plausible but incorrect rationales?

To answer these questions, we propose DyG-RoLLM¹, the first framework to provide interpretable dynamic graph clustering via bidirectional GNN-LLM interaction. **First**, to learn clustering-aware representation, we simultaneously learn clustering patterns and interpretable features through orthogonal subspace decomposition. GAT-GRU² captures structural-temporal dynamics, whose outputs are then separated into two orthogonal subspaces: one optimizes modularity for community detection, while the other learns features for semantic interpretation. This orthogonal design prevents the two objectives from interfering with each other. **Second**, to establish structure-semantic alignment, the interpretable subspace specifically learns five role prototypes (Leader, Contributor, Wanderer, Connector, Newcomer) that bridge continuous embeddings to discrete language. These prototypes transform node representations into role distributions (e.g., node v_{24} has 70% Leader traits), which generate hierarchical descriptions spanning node-level roles, community-level compositions, and evolution-level patterns. This multi-level semantic representation provides LLMs with comprehensive context for understanding each node's position, function, and trajectory within the dynamic graph. **Finally**, to provide reasoning guidance, we leverage these hierarchical descriptions in a chain-of-thought framework where LLMs analyze role-community compatibility, supply-demand imbalances, and temporal consistency to determine cluster assignments. Crucially, high-confidence LLM predictions serve as soft supervision for GNN training through consistency loss, while GNN provides structural context for LLM reasoning. This bidirectional loop enables semantic understanding to refine structural clustering and structural patterns to guide semantic interpretation, yielding both accurate clustering and faithful explanations. The main contributions can be summarized as:

- We formalize the interpretable dynamic graph clustering problem, and propose the first framework that generates natural language explanations alongside clustering decisions.
- We introduce role prototypes as semantic anchors that bridge neural and symbolic spaces, enabling the derivation of community and evolution interpretations from node-level roles.
- We establish a GNN-LLM interaction with CoT reasoning for dynamic graphs, validated via case studies showing how LLM reasoning corrects structural misclustering.
- Experimental results on ten benchmarks demonstrate that DyG-RoLLM achieves state-of-the-art clustering performance while providing human-interpretable explanations.

2 Related Work

Graph Neural Networks have gradually become the dominant paradigm for training graph models, and a wealth of outstanding works

has emerged accordingly [9, 25, 69, 72, 78–81, 92]. Current dynamic graph clustering methods can be mainly classified into three categories [29, 31, 32]: Neural Network (NN)-based methods, Matrix Factorization (MF)-based methods, and LLM-based methods. **First**, NN-based methods often employ a coupled strategy, where they first condense dynamic graphs into a single static graph and then apply clustering methods to identify clusters [57, 82, 91, 93]. Other NN-based methods employ two-stage strategies, where they first learn dynamic graph embeddings and then apply clustering methods on these embeddings to identify communities [1, 8, 11, 18, 41, 52, 73, 83, 90]. For example, RNNGCN [74] and DGCN [15] use RNNs or LSTMs to capture temporal dependencies for graph embeddings, which are then clustered using graph convolutional layers. ROLAND [76] extends static GNN-based graph embedding methods to dynamic graphs by using GRUs to capture temporal information. **Second**, MF-based methods perform matrix factorization at each timestamp to cluster nodes while enforcing temporal consistency across snapshots. These methods differ primarily in how they measure and maintain temporal smoothness. One category exploits topology changes: sE-NMF [45], jLMDC [30], and NE2NMF [33] analyze changes between consecutive graphs, while PisCES [36] considers global topology evolution. Another category directly optimizes clustering metrics: DynaMo [96] incrementally maximizes modularity, and MODPSO [75] minimizes cross-timestamp NMI. **Third**, LLM-based methods integrate GNNs with LLMs for graph understanding, following two primary paradigms depending on which component serves as the final predictor. *GNN-as-predictor* approaches [20, 21, 44, 63, 64, 88] enhance GNN inputs through LLM-processed features. For example, GraphGPT [62] enhances graph prediction by combining semantic embeddings from LLMs with structural embeddings from GNNs. GraphTranslator [85] employs LLMs to extract semantic features from text descriptions of graphs. UniGraph [21] uses pre-trained LLMs to encode textual node attributes. *LLM-as-predictor* methods [19, 54, 68, 70, 71] transform graph structure into natural language for direct LLM reasoning. For example, ComGPT [49] encodes adjacency information with iterative prompting, STC-CDP [95] combines structural descriptions with few-shot demonstrations, and CommLLM [17] converts entire graphs into text narratives. **In contrast to all existing methods**, DyG-RoLLM fundamentally reimagines dynamic graph clustering through three key innovations: *(i)* While existing methods produce black-box results, we generate natural language explanations for every clustering decision; *(ii)* Unlike unidirectional GNN-LLM pipelines, we establish bidirectional interaction where LLMs provide soft supervision for GNN training while GNNs guide LLM reasoning; *(iii)* Beyond simple feature enhancement, our learnable role prototypes create semantic bridges that enable reasoning about why communities form and how they evolve.

3 Preliminary

Dynamic Graph Clustering. We consider a dynamic graph as a sequence of snapshots $\mathcal{G} = \{G_1, G_2, \dots, G_\tau\}$ and the t -th snapshot $G_t = (V^t, E^t, X^t)$, defined for $0 \leq t \leq \tau$. Here, V^t , E^t and X^t represent the set of nodes, edges, and node features at t -th snapshot. Dynamic community detection aims to partition the node set V^t into non-overlapping clusters $C^t = \{C_1^t, \dots, C_{K_t}^t\}$ where K_t denotes

¹DyG-RoLLM denotes **D**ynamic **G**raph Clustering via **N**ode **R**ole-guided **L**LM.

²Graph Attention Network (GAT) encoder with a Gated Recurrent Unit (GRU) updater.

the number of clusters at t -th timestamp, $\bigcup_{k=1}^{K_t} C_k^t = V^t$, and $C_i^t \cap C_j^t = \emptyset$ for $i \neq j$. Beyond traditional clustering, our task learns the mapping $\Psi : G^t \rightarrow (C^t, \{\mathcal{E}_i\}_{i=1}^{|V^t|})$, where $\mathcal{E}_i$ represents hierarchical natural language explanations for node i 's community assignment.

4 Methodology

As shown in Figure 1, DyG-RoLLM consists of three components: (i) **Clustering-aware Graph Encoding** employs orthogonal subspace decomposition to separate role features from community features (Sec.4.1), (ii) **Interpretable Node-Roles Learning** defines semantic prototypes bridging graph embeddings with natural language (Sec.4.2), and (iii) **Hierarchical LLM Reasoning** performs structured CoT for interpretable clustering (Sec.4.3).

4.1 Clustering-Aware Graph Encoding

To capture evolving clustering patterns in dynamic graphs, we design a dynamic graph encoding module with **orthogonal subspace decomposition**. This module learns node representations that jointly encode node roles and clusters in independent subspaces, providing the foundation for interpretable graph clustering.

4.1.1 Dynamic Graph Encoding. Given a dynamic graph $\mathcal{G}$, we first encode the structural information at each timestep $G_t = \{V^t, E^t\}$ using a multi-layer GAT. For a node i at time t with feature vector $\mathbf{x}_i^t \in \mathbb{R}^d$, the l -th GAT layer computes the node representation as:

$$\mathbf{h}_i^{t,(l)} = \text{ReLU}\left(\sum_{j \in \mathcal{N}_i^t} \alpha_{ij}^{t,(l)} \mathbf{W}^{(l)} \mathbf{h}_j^{t,(l-1)}\right) \quad (1)$$

where $\mathbf{h}_i^{t,(0)} = \mathbf{x}_i^t$, $\mathcal{N}_i^t$ is the neighborhood of node i at timestep t , $\mathbf{W}^{(l)}$ is the trainable weight matrix, and $\alpha_{ij}^{t,(l)}$ can be computed as:

$$\alpha_{ij}^{t,(l)} = \frac{\exp\left(\text{LeakyReLU}\left(\mathbf{a}^\top [\mathbf{W}^{(l)} \mathbf{h}_i^{t,(l-1)} \parallel \mathbf{W}^{(l)} \mathbf{h}_j^{t,(l-1)}]\right)\right)}{\sum_{m \in \mathcal{N}_i^t \cup \{i\}} \exp\left(\text{LeakyReLU}\left(\mathbf{a}^\top [\mathbf{W}^{(l)} \mathbf{h}_i^{t,(l-1)} \parallel \mathbf{W}^{(l)} \mathbf{h}_m^{t,(l-1)}]\right)\right)}, \quad (2)$$

where $\parallel$ denotes concatenation and $\mathbf{a} \in \mathbb{R}^{2d}$ is a learnable vector. To capture temporal dependencies across snapshots, we employ a Gated Recurrent Unit (GRU) that maintains node states over time:

$$\mathbf{s}_i^t = \text{GRU}\left(\mathbf{h}_i^{t,(L)}, \mathbf{s}_i^{t-1}\right) \quad (3)$$

where $\mathbf{h}_i^{t,(L)}$ is the final output, and $\mathbf{s}_i^t \in \mathbb{R}^d$ represents the node embedding at timestep t . A fundamental conflict arises: nodes with similar roles should cluster together, yet may belong to different communities requiring separation. We solve this via *orthogonal subspace decomposition*, as guaranteed by the following proposition.

Proposition 1 (Gradient Isolation via Orthogonal Decomposition). *Given a decomposition function $\mathcal{F} : \mathbb{R}^d \rightarrow \mathbb{R}^{d_1} \times \mathbb{R}^{d_2}$ defined as $\mathcal{F}(\mathbf{x}) = [\mathbf{W}_1 \mathbf{x} \parallel \mathbf{W}_2 \mathbf{x}]$ where $\mathbf{W}_1$ and $\mathbf{W}_2$ are projection matrices satisfying the orthogonality constraint $\mathbf{W}_1 \mathbf{W}_2^\top = \mathbf{0}$. For a composite loss $\mathcal{L} = \mathcal{L}_1(\mathbf{W}_1 \mathbf{x}) + \mathcal{L}_2(\mathbf{W}_2 \mathbf{x})$, the following gradient isolation property holds: $\nabla_{\mathbf{W}_1} \mathcal{L}_1 = \mathbf{0}$ and $\nabla_{\mathbf{W}_2} \mathcal{L}_2 = \mathbf{0}$. This ensures that optimizing one objective does not interfere with the other, allowing both tasks to achieve their respective optima without compromise.*

Motivated by Proposition 1 (Appendix A), we decompose node representations $\mathbf{s}_i^t$ into orthogonal role and clustering subspaces as:

$$\mathbf{z}_i^t = [\mathbf{z}_i^{t,\text{role}} \parallel \mathbf{z}_i^{t,\text{comm}}] = [\mathbf{W}_{\text{role}} \cdot \mathbf{s}_i^t \parallel \mathbf{W}_{\text{comm}} \cdot \mathbf{s}_i^t] \quad (4)$$

s.t., $\mathbf{W}_{\text{role}} \mathbf{W}_{\text{comm}}^\top = \mathbf{0}_{d_r \times d_c}$

where $\mathbf{z}_i^{t,\text{role}} \in \mathbb{R}^{d_r}$ encodes role features, $\mathbf{z}_i^{t,\text{comm}} \in \mathbb{R}^{d_c}$ captures cluster affiliation, and $\mathbf{W}_{\text{role}} \in \mathbb{R}^{d_r \times d}$ and $\mathbf{W}_{\text{comm}} \in \mathbb{R}^{d_c \times d}$ are two learnable orthogonal projection matrix.

4.1.2 Modularity-Guided Clustering. With decomposed orthogonal representations, we now optimize node representations in the cluster subspace to obtain clustering-aware node features.

We first generate a soft cluster assignment matrix $\mathbf{C}^t \in \mathbb{R}^{|V^t| \times K_t}$ from the cluster subspace, where C_{ij}^t represents the probability that the i -th node belongs to the j -th cluster, which is formulated as:

$$C_{ij}^t = \frac{\exp((\mathbf{z}_i^{t,\text{comm}})^\top \boldsymbol{\mu}_j^t / \tau_c)}{\sum_{\ell=1}^{K_t} \exp((\mathbf{z}_i^{t,\text{comm}})^\top \boldsymbol{\mu}_\ell^t / \tau_c)}, \quad (5)$$

where $\boldsymbol{\mu}_\ell^t \in \mathbb{R}^{d_c}$ denotes the ℓ -th learnable cluster centroid, and τ_c is a temperature parameter that controls the sharpness of assignment.

Modularity-based Clustering Loss. Previous works [30, 40] directly maximize the modularity objective $Q_{\text{modul}} = \text{Tr}(\mathbf{C}^\top \mathbf{B} \mathbf{C})$, where $\mathbf{B} = \mathbf{A} - \frac{\mathbf{d} \mathbf{d}^\top}{2|E^t|}$ is the modularity matrix with $\mathbf{d}_i = \sum_j \mathbf{A}_{ij}$, to encourage densely connected nodes to cluster together while separating sparsely connected ones. However, this formulation requires $O(|V^t|^2)$ complexity, severely limiting the scalability to large graphs. To address this computational bottleneck, we reformulate modularity optimization through an efficient decomposition. By leveraging the algebraic structure of the modularity matrix, we derive an equivalent objective that avoids quadratic complexity as:

$$\mathcal{L}_{\text{mod}}^t = - \sum_{(i,j) \in E^t} \text{sim}(\mathbf{z}_i^{t,\text{comm}}, \mathbf{z}_j^{t,\text{comm}}) + \frac{1}{2|E^t|} \left\| \sum_{i \in V^t} \mathbf{d}_i \mathbf{z}_i^{t,\text{comm}} \right\|^2, \quad (6)$$

where the first term aggregates similarities over existing edges, encouraging connected nodes to have similar embeddings, and the second term acts as a degree-aware regularizer that prevents high-degree nodes from dominating the embedding space. This decomposition reduces the complexity from $O(|V^t|^2)$ to $O(|E^t| + |V^t|d_c)$, enabling efficient optimization on web-scale graphs while preserving the theoretical properties of modularity maximization.

Temporal Smoothing Loss. To ensure smooth cluster evolution while adapting to structural changes, we regularize assignment consistency for persistent nodes between successive snapshots as:

$$\mathcal{L}_{\text{temp}}^t = \frac{1}{|V^t \cap V^{t-1}|} \sum_{i \in V^t \cap V^{t-1}} \|\mathbf{C}_{i:}^t - \mathbf{C}_{i:}^{t-1}\|^2 \cdot \exp(-\alpha \cdot \delta_i) \quad (7)$$

where $\mathbf{C}_{i:}^t$ is node i 's cluster assignment, $\delta_i = \frac{\|\mathbf{A}_{i:}^t - \mathbf{A}_{i:}^{t-1}\|_1}{\|\mathbf{A}_{i:}^t\|_1 + \|\mathbf{A}_{i:}^{t-1}\|_1}$ measures local structural change and α controls adaptation sensitivity. This formulation allows for larger assignment changes for nodes with significant neighborhood modifications while maintaining stability.

Overall Clustering Objective. The complete loss function for cluster-aware representation learning can be formulated as:

$$\mathcal{L}_{\text{cluster}} = \mathcal{L}_{\text{mod}}^t + \lambda_1 \mathcal{L}_{\text{temp}}^t \quad \text{s.t., } \mathbf{W}_{\text{role}} \mathbf{W}_{\text{comm}}^\top = \mathbf{0}_{d_r \times d_c} \quad (8)$$

where λ_1 balances the trade-off between modularity maximization and temporal smoothness. This joint optimization ensures that learned clusters are topologically meaningful and temporally stable.

4.2 Interpretable Node-Roles Learning

This module establishes the node-role prototypes as semantic anchors, bridging the continuous graph embedding space and the

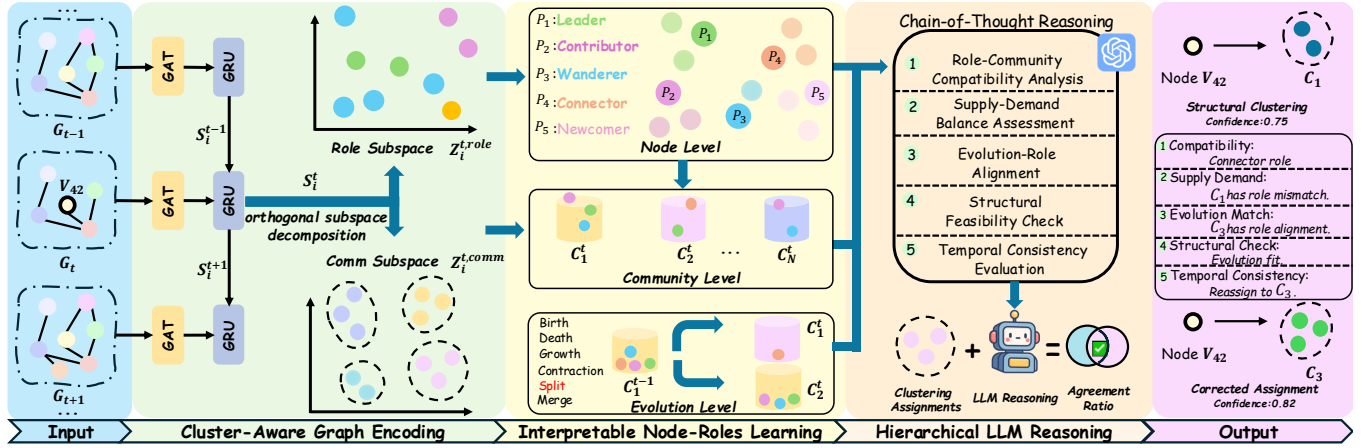


Figure 1: Overview of DyG-RoLLM. We first learn cluster-aware representation through orthogonal decomposition, then bridge them to semantics via node-roles prototypes, and finally enable hierarchical LLM reasoning for interpretable clustering.

discrete language space for LLM-driven inference. The prototypes are designed to be dynamic, robust, and intrinsically interpretable across node-level, community-level, and evolution-level scales.

4.2.1 Node Role Definition. With orthogonally decomposed representations that separate role from community features, we focus on the role subspace to define interpretable node-role prototypes that enable semantic understanding of clustering results.

Definition 1 (Node Roles). Drawing from *Structural Node Role Theory* [3, 47], we define five node prototypes $\mathbf{p}^{\text{node}} = \{\mathbf{p}_1, \dots, \mathbf{p}_5\}$ serving as functional roles with natural-language interpretation as:

- $\mathbf{p}_1$: [Leader] - High centrality nodes, influential hubs maintaining a *stable core set of internal connections*.
- $\mathbf{p}_2$: [Contributor] - Moderate connectivity nodes, consistently present, primarily forming *intra-community connections*.
- $\mathbf{p}_3$: [Wanderer] - Weakly connected nodes at boundaries with *low clustering coefficient* and *high migration potential*.
- $\mathbf{p}_4$: [Connector] - Inter-community connectors with *high betweenness centrality* and *low internal clustering coefficient*.
- $\mathbf{p}_5$: [Newcomer] - Recently joined nodes with *high connection volatility* and *rapid changes in structural metrics*.

Node Roles Initialization. To ensure meaningful role semantics, we initialize the i -th prototype as $\mathbf{f}_i^{\text{prior}} = [\mathbf{f}_i^{\text{C}}, \mathbf{f}_i^{\text{L}}, \mathbf{f}_i^{\text{T}}]$, where the centrality features $\mathbf{f}^{\text{C}}$ include degree, betweenness, and closeness centrality; local structure features $\mathbf{f}^{\text{L}}$ include the clustering coefficient and intra-community density; temporal volatility features $\mathbf{f}^{\text{T}}$ include connection volatility and role stability. These synthetic vectors encode ideal role features (details in Appendix B). We then use a shared Multi-Layer Perceptron (MLP_{θ_1}) to map the role-specific synthetic feature prior to the role representation subspace as:

$$\mathbf{p}_i = \text{MLP}_{\theta_1}(\mathbf{f}_i^{\text{prior}}) + \epsilon_i, \quad \text{where } \epsilon_i \sim \mathcal{N}(0, \sigma^2 \mathbf{I}). \quad (9)$$

4.2.2 Role-Embedding Alignment. To align node representations with the defined role prototypes, we employ contrastive learning in the role subspace. This mechanism enables unsupervised discovery of node roles without requiring manual annotations. Specifically,

for each node i with role representation $\mathbf{z}_i^{t,\text{role}} \in \mathbb{R}^{d_r}$, we calculate its affinity to each node-role prototype as follows:

$$\pi_{ik}^t = \frac{\exp(\text{sim}(\mathbf{z}_i^{t,\text{role}}, \mathbf{p}_k) / \tau_r)}{\sum_{j=1}^5 \exp(\text{sim}(\mathbf{z}_i^{t,\text{role}}, \mathbf{p}_j) / \tau_r)} \quad (10)$$

where $\text{sim}(\cdot, \cdot)$ denotes cosine similarity and τ_r is the temperature parameter. This soft assignment allows nodes to exhibit mixed roles, i.e., a node might be 70% Leader and 30% Connector, simultaneously. **Contrastive Prototype Loss.** We optimize prototypes and node representations jointly via a contrastive objective that attracts nodes to their dominant prototype while maintaining prototype diversity:

$$\mathcal{L}_{\text{proto}}^t = -\frac{1}{|V^t|} \sum_{v_i \in V^t} \log \pi_{ik^*}^t + \lambda_2 \sum_{k=1}^5 \hat{\pi}_k^t \log \hat{\pi}_k^t, \quad (11)$$

where $k^* = \arg \max_k \pi_{ik}^t$ and $\hat{\pi}_k^t = \frac{1}{|V^t|} \sum_{v_i \in V^t} \pi_{ik}^t$. The first term attracts nodes toward their nearest prototype, while the second term prevents prototype collapse and ensures all roles remain active.

4.2.3 Role-Semantic Alignment. The learned role prototypes bridge neural representations and natural language by generating interpretable descriptions at multiple granularities—from individual nodes to community compositions to temporal evolution patterns. **Community Composition.** For each detected community $\{C_k\}_{k=1}^{K_t}$, we derive its compositional signature by aggregating member roles:

$$\gamma_{C_k}^t = \frac{1}{|C_k^t|} \sum_{v_i \in C_k^t} \pi_i^t = [\gamma_{C_k,1}^t, \dots, \gamma_{C_k,5}^t] \quad (12)$$

where $\pi_i^t = [\pi_{i1}^t, \dots, \pi_{i5}^t]$ is node i 's role distribution. This composition γ^t directly reveals community features for LLM interpretation. **Evolution Pattern.** Temporal changes in role distributions naturally map to evolution patterns. Given two snapshots, we compute:

$$\Delta \gamma_{C_k}^t = \gamma_{C_k}^{t+1} - \gamma_{C_k}^t \quad (13)$$

to help LLMs understand six evolution events, including *Birth*, *Death*, *Growth*, *Contraction*, *Split*, *Merge* (details in Appendix C). **Natural Language Template Generation.** Each node can be converted to a semantic description by a parametric template function

$\mathcal{T}(\cdot)$, which is listed in Appendix D. For node v_i , we can generate the following semantic description via $\mathcal{T}(\pi_i^t, \gamma_C^t, \Delta\gamma_C^t, \mathcal{G}, C, \Theta)$, where $\mathcal{G}$ denotes dynamic graphs and Θ is the activation threshold:

Node Role-Guided Description

Node-level: Node v_{23} exhibits dominant Leader characteristics (70%) with secondary Contributor traits (20%). It maintains 45 connections with betweenness centrality 0.82, serving as a critical hub in the network structure.

Community-level: Node v_{23} belongs to Community C_3 (89 members) with composition [15% Leaders, 50% Contributors, 25% Wanderers, 8% Connectors, 2% Newcomers]. As a primary leader in this hierarchical core community, v_{23} coordinates with 12 leaders to maintain structural cohesion.

Evolution-level: Over the past timestep, the community v_{23} experienced a growth pattern with 18% newcomer influx while maintaining stable leadership. Node v_{23} 's role evolved from 55% to 70% Leader confidence, indicating consolidation of its leadership position during expansion phase.

4.3 Hierarchical LLM Reasoning

We employ LLMs to perform structured reasoning about community assignments, ensuring that detected communities are not only structurally coherent but also logically meaningful.

4.3.1 Structured Reasoning. Given the hierarchical semantic descriptions generated by function $\mathcal{T}(\cdot)$, LLMs perform cluster assignment via structured probabilistic reasoning:

$$Q^t = \text{LLM}_{\text{Reason}} \left(\mathcal{T}(\{\pi_i^t\}_{i \in V^t}, \{\gamma_{C_k}^t, \Delta\gamma_{C_k}^t\}_{k=1}^{K_t}, \mathcal{G}, C, \Theta) \right), \quad (14)$$

where Q^t denotes the assignment probabilities. We design a reasoning chain to determine optimal cluster assignments as:

Chain-of-Thought Reasoning for Dynamic Graphs

INPUT: Three-level semantic description from template $\mathcal{T}(\cdot)$

REASONING STEPS:

Step 1 - Role-Community Compatibility Analysis

Evaluate how node's role distribution aligns with each community's composition

Step 2 - Supply-Demand Balance Assessment

Identify imbalances in role distributions across communities

Step 3 - Evolution-Role Alignment

Match node's role trajectory with community evolution patterns

Step 4 - Structural Feasibility Check

Validate assignment with connectivity patterns

Step 5 - Temporal Consistency Evaluation

Consider historical assignments and migration costs

OUTPUT: Probability distribution $Q_{i,k}$ over K communities with justification

4.3.2 Consistency Learning. To align structural clustering and semantic reasoning, we quantify the agreement between clustering assignments C^t and LLM reasoning Q^t using:

$$\mathcal{A}^t = \frac{1}{|V^t|} \sum_{i \in V^t} \mathbb{I}[\arg \max_k C_{ik}^t = \arg \max_k Q_{ik}^t] \in [0, 1], \quad (15)$$

where $\mathcal{A}$ is the agreement ratio. Low agreement indicates structural patterns contradict semantic logic. Rather than using all LLM outputs, we selectively use high-confidence semantic insights as:

$$\mathcal{L}_{\text{consist}} = -\frac{1}{|\mathcal{V}_{\text{conf}}^t|} \sum_{i \in \mathcal{V}_{\text{conf}}^t} \sum_{k=1}^{K_t} Q_{ik}^t \log C_{ik}^t \quad (16)$$

where $\mathcal{V}_{\text{conf}}^t = \{i : \max_k Q_{ik}^t > \tau_{\text{conf}}\}$ contains nodes with confident LLM predictions (typically $\tau_{\text{conf}} = 0.8$). This prevents uncertain reasoning from destabilizing structural optimization.

Objective Function. The total optimization objective becomes:

$$\mathcal{L}_{\text{total}} = \mathcal{L}_{\text{cluster}} + \mathcal{L}_{\text{proto}} + \lambda_{\text{LLM}}(\mathcal{A}^t) \mathcal{L}_{\text{consist}} \quad (17)$$

where the adaptive weight $\lambda_{\text{LLM}}(\mathcal{A}^t) = \max(0, 1 - \mathcal{A}^t)$ increases LLM influence when disagreement is high.

4.3.3 Interpretable Outputting. The framework produces both final assignments and comprehensive explanations.

Final Assignment. We combine C^t with LLM reasoning Q^t as:

$$\hat{k}_i = \arg \max_k [\alpha C_{ik}^t + (1 - \alpha) Q_{ik}^t] \quad (18)$$

where $\alpha = 0.7$ balances structural coherence and semantic insights.

Explanation Generation. Each assignment produces explanations aligned with the five-step reasoning chain:

Final Explanation Generation

Assignment Decision: Node $v_{23} \rightarrow$ Community C_2

Step 1 - Compatibility: How node's roles fit community composition.

Example: Leader (70%) + Connector (20%) complements C_2 's role gaps.

Step 2 - Supply-Demand: Role distribution imbalances across communities.

Example: C_1 oversupplied (35% Leaders) vs C_2 undersupplied (5% Leaders).

Step 3 - Evolution Match: Node trajectory alignment with community dynamics.

Example: Rising Leader confidence (55% $\rightarrow$ 70%) matches C_2 's growth needs.

Step 4 - Structural Check: Connectivity validation for assignment.

Example: 12 of 45 connections in C_2 ; betweenness 0.82 supports transition.

Step 5 - Temporal Consistency: Historical assignment and migration feasibility.

Example: Connector trait (20%) enables smooth migration from C_1 to C_2 .

Final Confidence: 0.81 (Structural: 0.75, Semantic: 0.88).

4.4 Complexity Analysis

Time Complexity. The overall time complexity per snapshot is $O(L|E^t|d + |V^t|K_t d_c + |V^t|d_r)$, where L is the number of GAT layers, d, d_c, d_r is the dimensionality of the embedding space, cluster space, role space, respectively. K_t is number of clusters in the t -th timestep.

Space Complexity. The space requirement is $O(|V^t|d + K_t d_c)$. For web-scale deployment, embeddings can be computed in mini-batches, requiring only $O(bd)$ memory where b is batch size. We provide more details about complexity analysis in Appendix E.

Table 1: Detailed statistics of dynamic graph benchmarks.

Dynamic Graphs	# of Nodes	# of Edges	# of Snapshots
Birth-Death (BD)	30K & 100K	12M & 24M	10 & 20
Expand-Contract (EC)	30K & 100K	13M & 26M	10 & 20
Disappear-Reappear (DR)	30K & 100K	13M & 28M	10 & 20
Merge-Split (MS)	30K & 100K	14M & 29M	10 & 20
Wikipedia	8,400	162,000	5
Dublin	11,000	415,900	5
arXiv	28,100	4,600,000	5
Stack-Overflow (SO)	2,601,977	63,497,050	5
Flickr	2,302,925	33,100,000	5
Youtube	3,200,000	12,200,000	5

5 Experiment

Datasets. As shown in Table 1, following previous works [33, 77], we evaluate baselines on four synthetic dynamic graphs and six real-world dynamic graphs with varying number of nodes and edges. Synthetic Green datasets [14] generate dynamic graphs considering four evolution events including *Birth-Death* (BD): existing clusters are removed or generated by randomly selecting nodes from other clusters; *Expand-Contract* (EC): clusters are expanded or contracted; *Disappear-Reappear* (DR): clusters are randomly hidden and reappeared; *Merge-Split* (MS): clusters are split or merged.

Table 2: Overall performances on dynamic graphs. Bold and Underline indicate the best and second-best performing methods. Symbol $\dagger$ indicates that DyG-RoLLM significantly surpassed all baselines with a p-value < 0.005 . N/A means that it cannot be executed due to memory and running time constraints.

Methods	BD-30K		EC-30K		BD-100K		DR-100K		MS-100K		Wikipedia		Dublin		arXiv		SO		Flickr		Youtube		
	NMI	NF1	NMI	NF1	NMI	NF1	NMI	NF1	NMI	NF1	NMI	NF1	NMI	NF1	NMI	NF1	NMI	NF1	NMI	NF1	NMI	NF1	
NN-based	node2vec [16]	93.8	85.8	93.2	83.6	90.3	84.2	91.2	85.6	89.0	78.2	33.5	10.2	50.3	23.2	44.3	26.8	40.3	23.3	42.4	25.5	42.8	24.5
	RNNGCN [74]	96.6	84.5	95.9	85.1	90.8	84.9	92.1	84.3	91.2	80.9	40.3	22.5	52.2	31.6	45.4	26.2	40.8	23.8	48.5	30.2	47.6	32.3
	SepNE [35]	92.5	89.4	92.1	81.8	89.1	82.2	89.0	81.1	89.2	78.6	31.4	9.8	49.2	21.0	42.8	25.8	39.2	22.1	40.3	22.4	39.5	21.0
	ROLAND [76]	95.5	83.2	94.1	83.8	<u>92.8</u>	85.1	93.3	85.8	92.8	81.6	42.2	22.3	53.6	31.6	46.8	27.6	44.8	29.2	47.5	31.4	48.4	33.2
	TGC [39]	95.3	83.0	93.8	83.6	92.0	84.5	92.8	85.4	91.5	81.2	41.3	22.3	52.8	31.3	45.8	26.8	44.6	29.0	47.8	31.6	47.9	32.6
	DSCPCD [66]	89.2	69.8	88.9	70.3	81.2	65.8	79.6	63.5	81.0	64.4	28.2	7.3	32.4	11.9	31.8	10.9	34.4	18.6	34.3	18.2	32.6	15.9
MF-based	PisCES [36]	91.2	41.6	92.6	49.0	N/A	N/A	N/A	N/A	N/A	N/A	32.1	9.9	46.3	16.2	38.2	14.5	N/A	N/A	N/A	N/A	N/A	N/A
	DYNMOGA [14]	<u>98.1</u>	78.1	98.2	65.3	N/A	N/A	N/A	N/A	N/A	N/A	36.2	9.9	49.8	20.1	39.1	24.5	N/A	N/A	N/A	N/A	N/A	N/A
	NE2NMF [33]	97.1	76.1	97.5	63.1	N/A	N/A	N/A	N/A	N/A	N/A	34.1	8.2	47.9	18.9	38.2	22.9	N/A	N/A	N/A	N/A	N/A	N/A
	RTSC [77]	92.8	55.3	92.1	53.2	N/A	N/A	N/A	N/A	N/A	N/A	30.6	11.3	46.6	19.3	38.2	20.2	N/A	N/A	N/A	N/A	N/A	N/A
	RDMA [53]	95.3	69.8	94.8	85.5	N/A	N/A	N/A	N/A	N/A	N/A	33.8	10.2	47.2	18.6	41.6	25.2	N/A	N/A	N/A	N/A	N/A	N/A
	DyG-MF [28]	99.9	<u>90.2</u>	<u>99.2</u>	<u>90.9</u>	92.3	<u>86.8</u>	<u>94.3</u>	<u>86.5</u>	<u>94.4</u>	<u>83.2</u>	<u>50.4</u>	<u>25.8</u>	<u>56.1</u>	<u>33.7</u>	<u>51.8</u>	<u>30.2</u>	<u>45.5</u>	<u>29.6</u>	<u>52.3</u>	<u>33.6</u>	<u>51.8</u>	<u>34.5</u>
LLMs	ComGPT [49]	90.4	70.2	90.2	71.3	82.5	65.9	81.2	65.4	83.4	65.5	30.5	8.9	45.9	19.4	35.5	16.7	38.2	20.1	38.2	20.6	36.2	20.6
	CommLLM [17]	88.3	68.7	86.2	69.1	79.3	63.2	75.8	60.1	79.2	62.3	26.4	7.2	38.8	14.4	30.3	10.5	32.2	17.3	33.3	18.2	30.8	15.1
	DyG-RoLLM	99.9	90.4	99.4	91.7[†]	94.3[†]	90.9[†]	96.2[†]	88.6[†]	95.4[†]	84.8[†]	51.6[†]	26.7[†]	57.6[†]	35.2[†]	53.3[†]	32.1[†]	47.6[†]	30.9[†]	54.2[†]	34.8[†]	52.7[†]	35.3[†]

We also evaluate on various real-world domains, including *Academic Graphs*: arXiv [27]; *Social Graphs*: Dublin [22] and Flickr [46] and *Website Interaction Graphs*: Wikipedia [26], Stack-Overflow (SO) [51], and Youtube [46].

Baselines and Backbones. We compare DyG-RoLLM with 14 best-performing baselines, *i.e.*, Neural Network (NN)-based methods: node2vec [16], RNNGCN [74], SepNE [35], ROLAND [76], TGC [39], and DSCPCD [66]; and Matrix Factorization (MF)-based methods: PisCES [36], DYNMOGA [14], NE2NMF [33], RTSC [77], RDMA [53], and DyG-MF [28]; LLMs-based methods: ComGPT [49] and CommLLM [17].

Implementation Details. Following previous works [36, 45], we use normalized mutual information (NMI) [12] and normalized F1-score (NF1) [55] to measure clustering accuracy. We reproduced baselines using their optimal parameters and reported average performance over five repeated runs with different random seeds. We adopt GPT-4o as backbone LLM. We conducted multiple t-tests with Benjamini-Hochberg [2] correction to assess the statistical significance. We took DR-30K as validation set for hyperparameter tuning. With grid search, our method achieves the best results with embedding sizes $d = 512$, $d_r = 128$, $d_c = 384$; sharpness parameters $\tau_c = \tau_r = 0.5$; and regularization weights $\lambda_1 = 0.1$ and $\lambda_2 = 0.2$.

5.1 Performance Evaluation

As shown in Table 2, we report the performance comparison across synthetic and real-world dynamic graphs. DyG-RoLLM consistently achieves the highest NMI and NF1 scores, demonstrating the effectiveness of clustering-aware graph encoding, interpretable node-role learning, and hierarchical LLM reasoning. **First**, compared to NN-based methods RNNGCN and ROLAND, DyG-RoLLM improves NMI scores by 12.9% and 9.8% on Flickr and Youtube, respectively. This improvement stems from our joint optimization of node embeddings and modularity-based clustering, where orthogonal decomposition ensures that community-specific features are learned independently from role patterns, providing optimally separated representations for clustering tasks. **Second**, compared

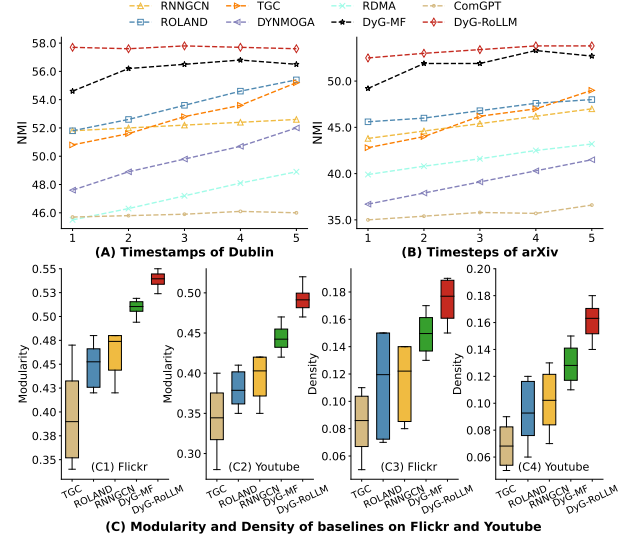


Figure 2: Performance comparison of (A-B) varying timestamps and (C) modularity and density of selected baselines.

to the best-performing MF-based method DyG-MF, DyG-RoLLM achieves 3% and 4% improvements in NMI and NF1 on real-world graphs. While MF-based methods produce opaque latent factors that conflate structural proximity with functional similarity, our method leverages interpretable role prototypes to guide LLM reasoning, ensuring both structural coherence through modularity optimization and semantic validity through logical consistency checks. **Third**, compared to LLM-based methods, DyG-RoLLM improves NMI by 48.64% over ComGPT and 66.92% over CommLLM on average. LLM methods rely solely on language models without grounding in graph structure, producing semantically plausible but topologically incorrect clusters. Our bidirectional learning ensures LLM reasoning is constrained by structural evidence while GNN

optimization benefits from semantic guidance, creating interaction that neither pure GNN nor LLM methods can achieve alone.

As shown in Figure 2 (A-B), we evaluate temporal performance across all timestamps. DyG-RoLLM demonstrates remarkable stability, maintaining superior performance throughout the entire evolution period with minimal variance, crucial for real-world deployment. Furthermore, we employ two label-free metrics, Modularity [48] and Conductance [7], to evaluate community quality without ground-truth dependency. This is essential because NMI and NF1 require potentially noisy or incomplete labels. Specifically, modularity measures intra-community edge density relative to random expectation, while conductance quantifies the cut quality between communities. Figure 2 (C) shows that DyG-RoLLM consistently outperforms the four best-performing baselines on both metrics across real-world graphs, validating its ability to identify structurally coherent communities independent of label quality.

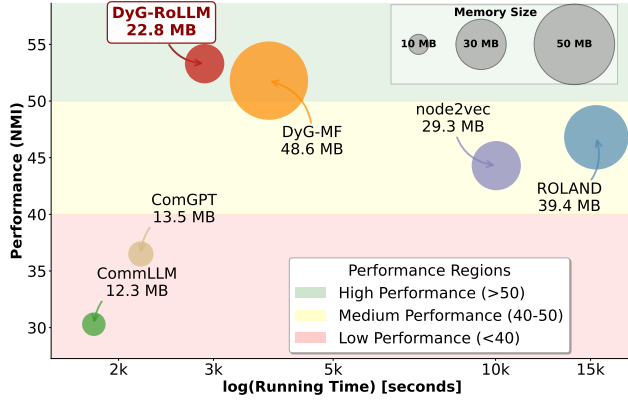


Figure 3: Efficiency comparison on large dynamic graphs.

5.2 Efficiency Evaluation

Figure 3 illustrates the efficiency-effectiveness trade-off across different baselines. DyG-RoLLM demonstrates a superior balance between computational efficiency and clustering performance. Despite the minimal parameter footprint of LLM-based approaches (ComGPT and CommLLM), their clustering accuracy remains inadequate for practical applications. While DyG-MF achieves competitive performance, its computational overhead and memory footprint limit scalability. Traditional neural approaches exhibit prohibitive resource consumption that precludes large-scale deployment. DyG-RoLLM uniquely combines high clustering accuracy with reasonable computational requirements, making it suitable for real-world applications. These results, showed on the medium-scale arXiv dataset, generalize consistently across all evaluated datasets.

5.3 Interpretability Evaluation

To validate our interpretable design, we conduct comprehensive evaluations through quantitative metrics and human assessment.

5.3.1 Quantitative Metrics. We evaluate interpretability through two metrics. First, we define Role Consistency Score (RCS) that

Table 3: Interpretability on real-world dynamic graphs.

Method	Wikipedia		Dublin		SO	
	RCS↑	EFS↑	RCS↑	EFS↑	RCS↑	EFS↑
ComGPT (+CoT)	0.69	0.58	0.64	0.55	0.71	0.60
CommLLM (+CoT)	0.62	0.51	0.58	0.47	0.65	0.53
DyG-RoLLM (w/o Role)	0.75	0.66	0.68	0.68	0.83	0.82
DyG-RoLLM (w/o CoT)	0.85	0.64	0.82	0.66	0.87	0.68
DyG-RoLLM	0.88	0.89	0.88	0.90	0.92	0.91

measures whether node roles remain stable over time as:

$$RCS = 1 - \frac{1}{|V^t \cap V^{t+1}|(T-1)} \sum_{i \in V^t \cap V^{t+1}} \sum_{t=2}^T \|\pi_i^t - \pi_i^{t-1}\|_1, \quad (19)$$

where RCS=1 denotes stable and meaningful roles.

Second, we define Explanation Fidelity Score (EFS) to verify whether explanations match actual graph structure as:

$$EFS = \frac{1}{N} \sum_{i=1}^N \frac{|\text{verified claims}_i|}{|\text{total claims}_i|} \quad (20)$$

where N denotes the number of randomly selected structural claims and we verify them against the graph. EFS=0.9 means 90% of explanatory claims are correct.

Since baselines lack interpretability, we adapt them for fair comparison. For **ComGPT (+CoT)** and **CommLLM (+CoT)**, we enhance them with CoT prompting to generate step-by-step reasoning about role and cluster assignments. For **DyG-RoLLM (w/o Role)**, we remove role description but retain LLM reasoning. **DyG-RoLLM (w/o CoT)** keeps roles but uses direct assignment without CoT reasoning. As shown in Table 3, DyG-RoLLM achieves superior interpretability by two key advantages. First, our method significantly outperforms LLM-only baselines on RCS because explicit role prototypes provide stable semantic anchors that prevent random explanation fluctuations. Second, DyG-RoLLM attains EFS>0.89, while enhanced baselines remain below 0.7. This gap reveals that pure LLM methods generate plausible-sounding but factually incorrect explanations. The performance drop in our ablations confirms that both role prototypes and structured reasoning are essential.

Table 4: Human evaluation with average scores on 1-5 scale.

Method	Clarity	Soundness	Complete	Trust	Action
ComGPT (+CoT)	3.2	2.8	2.6	2.9	2.5
CommLLM (+CoT)	2.9	2.5	2.4	2.6	2.3
DyG-RoLLM (w/o Role)	3.5	3.3	3.1	3.4	3.2
DyG-RoLLM (w/o CoT)	3.8	3.6	3.4	3.7	3.5
DyG-RoLLM (full)	4.4	4.5	4.3	4.6	4.2

5.3.2 Human Evaluation. Three domain experts (doctoral students in graph mining) independently evaluated 50 randomly selected explanations on Wikipedia using a 5-point Likert scale (1=poor, 5=excellent) across five criteria: **Clarity** (comprehensibility), **Soundness** (logical validity), **Completeness** (coverage of relevant factors), **Trustworthiness** (decision confidence), and **Actionability** (intervention insights). Table 4 shows that DyG-RoLLM achieves higher scores on all criteria, with Clarity=4.4 and Soundness=4.5, approaching "excellent" ratings. The high trustworthiness indicates

that experts can trust these explanations for decision making. Enhanced baseline score below 3.2, confirming that LLM reasoning without structural grounding produces less reliable explanations.

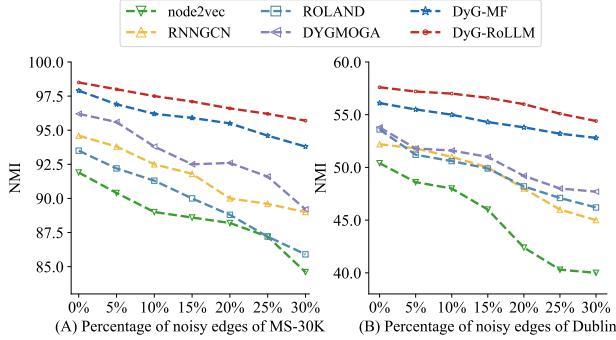


Figure 4: Noise Attacks. NMI w.r.t. percentage of noisy edges.

5.4 Robustness Evaluation

Real-world dynamic graphs often contain noise and exhibit irregular evolution patterns. To evaluate robustness, we contaminate graphs by injecting 5%-30% random edges per snapshot following [61]. As shown in Figure 4, DyG-RoLLM demonstrates superior noise resilience in both datasets. On MS-30K/Dublin, DyG-RoLLM experiences only 2.8%/5.6% NMI degradation in 30% noise compared to 4.2%/5.9% for DyG-MF. This robustness stems from three key designs: (1) orthogonal decomposition isolates role learning from noisy community boundaries, preventing error propagation; (2) role prototypes provide stable semantic anchors that filter spurious variations; (3) LLM reasoning validates assignments through logical consistency, rejecting noise-induced implausible structures. The consistent superiority across varying noise levels confirms DyG-RoLLM’s practical applicability in noisy real-world scenarios.

Table 5: Ablation and strategy replacement of DyG-RoLLM.

	Methods	Wikipedia		arXiv	
		NMI↑	NF1↑	NMI↑	NF1↑
Remove	w/o Orthogonal Subspace	42.2	22.4	43.5	15.8
	w/o Clustering loss Eq.(6)	50.3	25.6	51.6	30.0
	w/o Temporal loss Eq.(7)	51.0	26.3	52.5	31.7
	w/o Contrastive loss Eq.(11)	50.6	25.7	51.7	30.1
	w/o Consistent loss Eq.(16)	50.0	25.1	51.2	29.7
	w/o Role-Guided Description	48.3	24.1	46.2	16.4
	w/o LLM Reasoning	47.2	23.8	45.1	16.0
Replace	Random Prompt Init	51.0	26.4	52.9	32.0
	GPT-3.5-turbo	50.3	25.9	52.0	31.4
	Claude-3.5	50.8	26.0	52.6	32.0
	Llama2-7B	50.1	25.8	51.9	31.2
	DyG-RoLLM (full)	51.6	26.7	53.3	32.1

5.5 Ablation Study

We conduct comprehensive ablation studies to evaluate the necessity of each component in DyG-RoLLM. we adopt seven variants: *w/o Orthogonal Subspace* denotes using a unified embedding for both

clustering and prototype learning instead of separate subspaces; *w/o Role-guided Description* removes the three-level semantic descriptions for each node; and *w/o LLM Reasoning* bypasses CoT reasoning and directly predicts community assignments. The remaining variants ablate individual loss functions to assess their contributions. As shown in Table 5, removing any of these components negatively impacts performance on graph clustering, demonstrating their effectiveness and necessity. Furthermore, we conduct four strategy replacement experiments using random prototype initialization and different LLM backbones. The results demonstrate remarkable robustness: performance degradation remains minimal across all alternatives, indicating that DyG-RoLLM is not overly dependent on specific implementation choices and can be adapted to various deployment scenarios with different computational constraints.

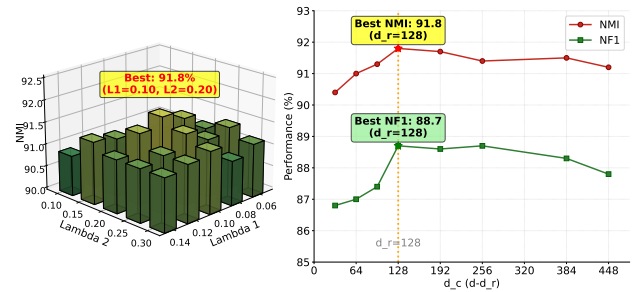


Figure 5: Hyperparameter tuning results.

5.6 Hyperparameter Sensitivity Analysis

We conducted sensitivity analysis for critical hyperparameters λ_1 and λ_2 , as well as the dimensions of cluster space d_c and role space d_r on the DR-30K dataset. As shown in Figure 5, DyG-RoLLM achieves optimal performance at $\lambda_1 = 0.1$ and $\lambda_2 = 0.2$. Increasing either parameter, which amplifies the weights of temporal and diversity components respectively, degrades model performance. These values were fixed for all experiments. Regarding dimensional allocation, the model performs best with $d_r = 128$ and $d_c = 384$, suggesting that cluster representation requires more features than role description. Notably, performance remains stable across various dimensional combinations, demonstrating the robustness of our approach and its practical applicability across different settings.

6 Conclusion

In this paper, we presented DyG-RoLLM, a framework for interpretable dynamic graph clustering. By introducing orthogonal subspace decomposition to separate role identification from clustering, we solve the conflict between these tasks. Through learnable prototypes as semantic anchors, we transform graph structures into interpretable narratives enabling LLM reasoning about communities. Future work will explore continuous-time dynamic graphs, incorporating multimodal node attributes.

Acknowledgment

This work was supported by the UTokyo Global Activity Support Program for Young Researchers. We sincerely thank our peers and reviewers for their valuable feedback to enhance our work.

References

- [1] Anna Beer, Andrew Draganov, Ellen Hohma, Philipp Jahn, Christian MM Frey, and Ira Assent. 2023. Connecting the Dots–Density–Connectivity Distance unifies DBSCAN, k-Center and Spectral Clustering. In *Proc. of SIGKDD*. 80–92.
- [2] Yoav Benjamini and Yoel Hochberg. 1995. Controlling the false discovery rate: a practical and powerful approach to multiple testing. *J R Stat Soc B* 57, 1, 289–300.
- [3] Stephen P Borgatti and Martin G Everett. 2000. Models of core/periphery structures. *Social networks* 21, 4 (2000), 375–395.
- [4] D. Chakrabarti, R. Kumar, and A. Tomkins. 2006. Evolutionary clustering. In *Proc. of SIGKDD*. 554–560.
- [5] Jie Chen, Licheng Jiao, Xu Liu, Lingling Li, Fang Liu, Puhua Chen, Shuyuan Yang, and Biao Hou. 2024. Hierarchical Dynamic Graph Clustering Network. *IEEE Trans. Knowl. Data Eng.* 36, 9 (2024), 4722–4735.
- [6] Jialin Chen and Rex Ying. 2023. Tempme: Towards the explainability of temporal graph neural networks via motif discovery. *Proc. of NeurIPS* 36, 29005–29028.
- [7] Mingming Chen, Tommy Nguyen, and K Szymanski. 2013. On measuring the quality of a network community structure. In *Proc. of SocialCom*. 122–127.
- [8] M Chen, C Wang, Dong Huang, Jian-Huang Lai, and Philip S Yu. 2022. Efficient orthogonal multi-view subspace clustering. In *Proc. of SIGKDD*. 127–135.
- [9] Yankai Chen, Yixiang Fang, Yifei Zhang, Chenhao Ma, Yang Hong, and Irwin King. 2024. Towards effective top-n hamming search via bipartite graph contrastive hashing. *IEEE Transactions on Knowledge and Data Engineering* (2024).
- [10] Zheng Chu, Jingchang Chen, Qianglong Chen, Weijiang Yu, and et al. 2024. Navigate through Enigmatic Labyrinth A Survey of Chain of Thought Reasoning: Advances, Frontiers and Future. In *Proc. of ACL*. 1173–1203.
- [11] Zeyu Cui, Zekun Li, Shu Wu, Xiaoyu Zhang, Qiang Liu, Liang Wang, and M Ai. 2024. DyGCN: Efficient Dynamic Graph Embedding With Graph Convolutional Network. *IEEE Trans. Neural Networks Learn. Syst.* 35, 4 (2024), 4635–4646.
- [12] Leon Danon, A Guileria, Jordi Duch, and Alex Arenas. 2005. Comparing community structure identification. *J STAT MECH-THEORY E* 05, 09 (2005), P09008.
- [13] Xiao Dong, Lei Zhu, Xueming Song, Jingjing Li, and Zhiyong Cheng. 2018. Adaptive Collaborative Similarity Learning for Unsupervised Multi-view Feature Selection. In *Proc. of IJCAI*. 2064–2070.
- [14] Francesco Folino and Clara Pizzuti. 2014. An evolutionary multiobjective approach for community discovery in dynamic networks. *IEEE Trans. Knowl. Data Eng.* 26, 8 (2014), 1838–1852.
- [15] Chao Gao, Junyou Zhu, Fan Zhang, Zhen Wang, and Xuelong Li. 2023. A Novel Representation Learning for Dynamic Graphs Based on Graph Convolutional Networks. *IEEE Trans. Cybern.* 53, 6 (2023), 3599–3612.
- [16] Aditya Grover and Jure Leskovec. 2016. node2vec: Scalable feature learning for networks. In *Proc. of SIGKDD*. 855–864.
- [17] Ekta Gujral and Apurva Sinha. 2025. LLMs Between the Nodes: Community Discovery Beyond Vectors. *arXiv preprint arXiv:2507.22955* (2025).
- [18] Xingzhi Guo, Baojian Zhou, and Steven Skiena. 2022. Subset node anomaly tracking over large dynamic graphs. In *Proc. of SIGKDD*. 475–485.
- [19] Zirui Guo, Lianghao Xia, Yanhua Yu, Yuling Wang, Kangkang Lu, Zhiyong Huang, and Chao Huang. 2024. Graphedit: Large language models for graph structure learning. *arXiv preprint arXiv:2402.15183* (2024).
- [20] X He, Y Tian, Y Sun, N Chawla, T Laurent, Y LeCun, X Bresson, and B Hooi. 2024. G-retriever: Retrieval-augmented generation for textual graph understanding and question answering. *Proc. of NeurIPS* 37, 132876–132907.
- [21] Yufei He and Bryan Hooi. 2024. Unigraph: Learning a cross-domain graph foundation model from natural language. *CoRR* (2024).
- [22] Lorenzo Isella, Juliette Stehlé, et al. 2011. Analysis of face-to-face behavioral networks. *Journal of The Bio. Bio.* 271, 1 (2011), 166–180.
- [23] S Ji, M Liu, L Sun, C Liu, and T Zhu. 2024. MemMap: An Adaptive and Latent Memory Structure for Dynamic Graph Learning. In *Proc. of SIGKDD*. 1257–1268.
- [24] Shuo Ji, Xiaodong Lu, Mingzhe Liu, Leilei Sun, Chuanren Liu, Bowen Du, and Hui Xiong. 2023. Community-based Dynamic Graph Learning for Popularity Prediction. In *Proc. of SIGKDD*. 930–940.
- [25] Lingbai Kong, Hanchen Yang, Wengen Li, Yichao Zhang, Jihong Guan, and Shuigeng Zhou. 2024. Traffexplaner: a framework toward gnn-based interpretable traffic prediction. *IEEE Transactions on Artificial Intelligence* 6, 3 (2024), 559–573.
- [26] J Leskovec, D Huttenlocher, and J Kleinberg. 2010. Governance in social media: A case study of the Wikipedia promotion process. In *Proc. of ICWSM*. 98–105.
- [27] Jure Leskovec, Jon M. Kleinberg, and Christos Faloutsos. 2005. Graphs over time: densification laws, shrinking diameters and possible explanations. In *Proc. of SIGKDD*. 177–187.
- [28] D Li, S Kosugi, Y Zhang, Manabu Okumura, Feng Xia, and Renhe Jiang. 2025. Revisiting Dynamic Graph Clustering via Matrix Factorization. In *Proc. of WWW*.
- [29] Dongyuan Li, Qiang Lin, and Xiaoke Ma. 2021. Identification of dynamic community in temporal network via joint learning graph representation and nonnegative matrix factorization. *Neurocomputing* 435 (2021), 77–90.
- [30] Dongyuan Li, Xiaoke Ma, and Maoguo Gong. 2023. Joint Learning of Feature Extraction and Clustering for Large-Scale Temporal Networks. *IEEE Transactions on Cybernetics* 53, 3 (2023), 1653–1666.
- [31] Dongyuan Li, Shiyin Tan, Ying Zhang, Ming Jin, Shirui Pan, Manabu Okumura, and Renhe Jiang. 2025. DyG-Mamba: Continuous State Space Modeling on Dynamic Graphs. In *Proc. of NeurIPS*.
- [32] Dongyuan Li, Shuyao Zhang, and Xiaoke Ma. 2022. Dynamic Module Detection in Temporal Attributed Networks of Cancers. *IEEE/ACM Transactions on Computational Biology and Bioinformatics* 19, 4 (2022), 2219–2230.
- [33] Dongyuan Li, Xiaoxiong Zhong, Zengfa Dou, Maoguo Gong, and Xiaoke Ma. 2021. Detecting dynamic community by fusing network embedding and nonnegative matrix factorization. *Knowl. Based Syst.* 221 (2021), 106961.
- [34] Yicong Li, Yu Yang, Jiannong Cao, Shuaiqi Liu, Haoran Tang, and Guandong Xu. 2024. Toward Structure Fairness in Dynamic Graph Embedding: A Trend-aware Dual Debiasing Approach. In *Proc. of SIGKDD*. 1701–1712.
- [35] Z. Li, L. Zhang, and G. Song. 2019. Sepne: Bringing separability to network embedding. In *Proc. of AAAI*. 4261–4268.
- [36] Fuchen Liu, David Choi, Lu Xie, and Kathryn Roeder. 2018. Global spectral clustering in dynamic networks. *PNAS* 115, 5 (2018), 927–932.
- [37] F Liu, J Wu, S Xue, C Zhou, J Yang, and Q Sheng. 2019. Detecting the evolving community structure in dynamic social networks. In *Proc. of WWW*. 1–19.
- [38] Fanzhen Liu, Jia Wu, Chuan Zhou, and Jian Yang. 2019. Evolutionary Community Detection in Dynamic Social Networks. In *Proc. of IJCNN*. 1–7.
- [39] Meng Liu, Yue Liu, Ke Liang, Wenxuan Tu, Siwei Wang, Sihang Zhou, and Xinwang Liu. 2024. Deep Temporal Graph Clustering. In *Proc. of ICLR*.
- [40] Y Liu, J Li, Y Chen, R Wu, E Wang, J Zhou, S Tian, S Shen, X Fu, C Meng, W Wang, and L Chen. 2024. Revisiting Modularity Maximization for Graph Clustering: A Contrastive Learning Perspective. In *Proc. of SIGKDD*. 1968–1979.
- [41] Bin Lu, Xiaoying Gan, Weinan Zhang, Huaxiu Yao, Luoyi Fu, and Xinbing Wang. 2022. Spatio-Temporal Graph Few-Shot Learning with Cross-City Knowledge Transfer. In *Proc. of SIGKDD*. 1162–1172.
- [42] Kirill Lukyanov, Georgii Sazonov, Serafim Boyarsky, and Ilya Makarov. 2025. Robustness questions the interpretability of graph neural networks: what to do? *arXiv preprint arXiv:2505.02566* (2025).
- [43] Huixin Ma, Kai Wu, Handing Wang, and Jing Liu. 2024. Higher Order Knowledge Transfer for Dynamic Community Detection With Great Changes. *IEEE Trans. Evol. Comput.* 28, 1 (2024), 90–104.
- [44] Qiyao Ma, Xubin Ren, and Chao Huang. 2024. Xrec: Large language models for explainable recommendation. *arXiv preprint arXiv:2406.02377* (2024).
- [45] Xiaoke Ma and Di Dong. 2017. Evolutionary nonnegative matrix factorization algorithms for community detection in dynamic networks. *IEEE Trans. Data. Eng.* 29, 5 (2017), 1045–1058.
- [46] Alan Mislove, Hema Swetha Koppula, et al. 2008. Growth of the Flickr Social Network. In *Proc. of WOSN*. 25–30.
- [47] Mark Newman. 2018. *Networks*. Oxford university press.
- [48] M. E. J. Newman. 2006. Modularity and community structure in networks. *PNAS* 103, 23 (2006), 8577–8582.
- [49] Li Ni, Haowen Shen, Lin Mu, Yiwen Zhang, and Wenjian Luo. 2024. Comgpt: Detecting local community structure with large language models. *arXiv preprint arXiv:2408.06658* (2024).
- [50] Michael Ostroski, Geoffrey Sanders, Trevor Steil, and Roger Pearce. 2025. Scalable edge clustering of dynamic graphs via weighted line graphs. *Journal of Complex Networks* 13, 3 (2025), 1–26.
- [51] Ashwin Paranjape, Austin R Benson, and Jure Leskovec. 2017. Motifs in temporal networks. In *Proc. of WSDM*. 601–610.
- [52] N Park, R Rossi, E Koh, I A Burhanuddin, Sungchul Kim, Fan Du, Nesreen Ahmed, and Christos Faloutsos. 2022. CGC: Contrastive Graph Clustering for Community Detection and Tracking. In *Proc. of WWW*. 1115–1126.
- [53] Somayeh Ranjesh, Behrooz Masoumi, and Seyyed Mohsen Hashemi. 2024. A novel robust memetic algorithm for dynamic community structures detection in complex networks. *Proc. of WWW* 27, 1 (2024), 3.
- [54] Xubin Ren, Wei Wei, Lianghao Xia, Lixin Su, Suqi Cheng, Junfeng Wang, Dawei Yin, and Chao Huang. 2024. Representation learning with large language models for recommendation. In *Proc. of WWW*. 3464–3475.
- [55] Giulio Rossetti. 2017. RDYN: graph benchmark handling community dynamics. *Journal of Complex Networks* 5, 6 (2017), 893–912.
- [56] Giulio Rossetti and Rémy Cazabet. 2018. Community Discovery in Dynamic Networks: A Survey. *ACM Comput. Surv.* 51, 2 (2018), 35:1–35:37.
- [57] Aniello De Santo, Antonio Galli, Vincenzo Moscato, and Giancarlo Sperli. 2021. A deep learning approach for semi-supervised community detection in Online Social Networks. *Knowl. Based Syst.* 229 (2021), 107345.
- [58] Chuan Shi, Junze Chen, Jiawei Liu, and Cheng Yang. 2024. Graph foundation model. *Frontiers of Computer Science* 18, 6 (2024), 186355.
- [59] Jiajie Su, Chaochao Chen, Weiming Liu, Fei Wu, Xiaolin Zheng, and Haoming Lyu. 2023. Enhancing Hierarchy-Aware Graph Networks with Deep Dual Clustering for Session-based Recommendation. In *Proc. of WWW*. 165–176.
- [60] Haoxin Sun, Xiaotian Zhou, and Zhongzhi Zhang. 2024. Fast Computation for the Forest Matrix of an Evolving Graph. In *Proc. of SIGKDD*. 2755–2764.
- [61] S Tan, J You, and D Li. 2022. Temporality- and Frequency-aware Graph Contrastive Learning for Temporal Network. In *Proc. of CIKM*. 1878–1888.

- [62] J Tang, Y Yang, W Wei, L Shi, L Su, S Cheng, D Yin, and C Huang. 2024. Graphgpt: Graph instruction tuning for large language models. In *Proc. of SIGIR*. 491–500.
- [63] Jiabin Tang, Yuhao Yang, Wei Wei, Lei Shi, Long Xia, Dawei Yin, and Chao Huang. 2024. Hight: Heterogeneous graph language model. In *Proc. of SIGIR*. 2842–2853.
- [64] Yijun Tian, Huan Song, Zichen Wang, Haozhu Wang, Ziqing Hu, Fang Wang, Nitesh V Chawla, and Panpan Xu. 2024. Graph neural prompting with large language models. In *Proc. of AAAI*, Vol. 38. 19080–19088.
- [65] C Tran-Kiem, M Paredes, Amanda C Perofsky, et al. 2025. Fine-scale patterns of SARS-CoV-2 spread from identical pathogen sequences. *Nature* (2025), 1–10.
- [66] Yuyao Wang, Jie Cao, Zhan Bu, Jia Wu, and Youquan Wang. 2023. Dual Structural Consistency Preserving Community Detection on Social Networks. *IEEE Trans. Knowl. Data Eng.* (2023), 1–14.
- [67] Zhen Wang, Chunyu Wang, Xianghua Li, Chao Gao, Xuelong Li, and Junyou Zhu. 2022. Evolutionary Markov Dynamics for Network Community Detection. *IEEE Trans. Knowl. Data Eng.* 34, 3 (2022), 1206–1220.
- [68] Wei Wei, Xubin Ren, Jiabin Tang, Qinyong Wang, Lixin Su, Suqi Cheng, Junfeng Wang, Dawei Yin, and Chao Huang. 2024. Llmrec: Large language models with graph augmentation for recommendation. In *Proc. of WSDM*. 806–815.
- [69] Feng Xia, Ciyuan Peng, Jing Ren, Falih Gozi Febrinanto, Renqiang Luo, Vidya Saikrishna, Shuo Yu, and Xiangjie Kong. 2025. Graph Learning. *Foundations and Trends® in Signal Processing* 19, 4 (2025), 371–551.
- [70] Lianghao Xia, Ben Kao, and Chao Huang. 2024. Opengraph: Towards open graph foundation models. *arXiv preprint arXiv:2403.01121* (2024).
- [71] H Xie, D Zheng, J Ma, H Zhang, V Ioannidis, X Song, Q Ping, et al. 2023. Graph-aware language model pre-training on a large graph corpus can help multiple graph applications. In *Proc. of SIGKDD*. 5270–5281.
- [72] Hanchen Yang, Jiaqi Wang, Jiannong Cao, Wengen Li, Jialun Zheng, Yangning Li, Chunyu Miao, Jihong Guan, Shuigeng Zhou, and Philip S Yu. 2025. OKG-LLM: Aligning ocean knowledge graph with observation data via LLMs for global sea surface temperature prediction. *arXiv preprint arXiv:2508.00933* (2025).
- [73] R Yang, Y Wu, X Lin, Q Wang, T Chan, and J Shi. 2024. Effective Clustering on Large Attributed Bipartite Graphs. In *Proc. of SIGKDD*. 3782–3793.
- [74] Yuhang Yao and Carlee Joe-Wong. 2021. Interpretable Clustering on Dynamic Graphs with Recurrent Graph Neural Networks. In *Proc. of AAAI*. 4608–4616.
- [75] Y Yin, Y Zhao, H Li, and X Dong. 2021. Multi-objective evolutionary clustering for large-scale dynamic community detection. *Inf. Sci.* 549 (2021), 269–287.
- [76] Jiaxuan You, Tianyu Du, and Jure Leskovec. 2022. ROLAND: Graph Learning Framework for Dynamic Graphs. In *Proc. of SIGKDD*. 2358–2366.
- [77] Jingyi You, Chenlong Hu, et al. 2021. Robust Dynamic Clustering for Temporal Networks. In *Proc. of CIKM*. 2424–2433.
- [78] Shuo Yu, Huafei Huang, Yanming Shen, Pengfei Wang, Qiang Zhang, Ke Sun, and Honglong Chen. 2024. Formulating and representing multiagent systems with hypergraphs. *IEEE Transactions on Neural Networks and Learning Systems* 36, 3 (2024), 4599–4613.
- [79] Shuo Yu, Yicong Li, Shuo Wang, Tao Tang, Qiang Zhang, Jingjing Zhou, Ivan Lee, and Feng Xia. 2025. CaGE: A causality-inspired graph neural network explainer for recommender systems. *ACM Transactions on Information Systems* 43, 6 (2025), 1–29.
- [80] Shuo Yu, Yingbo Wang, Ruolin Li, Guchun Liu, Yanming Shen, Shaoxiong Ji, Bowen Li, Fengling Han, Xiuzhen Zhang, and Feng Xia. 2025. Graph2text or graph2token: A perspective of large language models for graph learning. *ACM Transactions on Information Systems* (2025).
- [81] Shuo Yu, Yingbo Wang, Zhitao Wan, Yanming Shen, Qiang Zhang, and Feng Xia. 2025. Path-aware Few-shot Knowledge Graph Completion. *IEEE Transactions on Artificial Intelligence* (2025).
- [82] Han Zhang, Feiping Nie, and Xuelong Li. 2023. Large-Scale Clustering With Structured Optimal Bipartite Graph. *IEEE Trans. Pattern Anal. Mach. Intell.* 45, 8 (2023), 9950–9963.
- [83] Kaike Zhang, Qi Cao, et al. 2023. DyTed: Disentangled Representation Learning for Discrete-time Dynamic Graph. In *Proc. of SIGKDD*. 3309–3320.
- [84] Kaike Zhang, Qi Cao, Gaolin Fang, Bingbing Xu, Hongjian Zou, Huawei Shen, and Xueqi Cheng. 2023. Dyted: Disentangled representation learning for discrete-time dynamic graph. In *Proc. of SIGKDD*. 3309–3320.
- [85] M Zhang, M Sun, P Wang, S Fan, Y Mo, X Xu, H Liu, C Yang, and C Shi. 2024. Graphtranslator: Aligning graph model to large language model for open-ended tasks. In *Proc. of WWW*. 1003–1014.
- [86] Yanfu Zhang, Hongchang Gao, Jian Pei, and Heng Huang. 2022. Robust self-supervised structural graph neural network for social network prediction. In *Proc. of WWW*. 1352–1361.
- [87] Yao Zhang, Yun Xiong, Yongxiang Liao, Yiheng Sun, Yucheng Jin, Xuehao Zheng, and Yangyong Zhu. 2023. TIGER: Temporal Interaction Graph Embedding with Restarts. In *Proc. of WWW*. 478–488.
- [88] Haiteng Zhao, Shengchao Liu, Ma Chang, Hannan Xu, Jie Fu, Zhihong Deng, Lingpeng Kong, and Qi Liu. 2023. Gimlet: A unified graph-text model for instruction-based molecule zero-shot learning. *Proc. of NeurIPS* 36 (2023), 5850–5887.
- [89] Kesen Zhao and Liang Zhang. 2024. Causality-inspired spatial-temporal explanations for dynamic graph neural networks. In *Proc. of ICLR*.
- [90] P Zhao, H Yang, D Dey, and G Hu. 2023. Spatial clustering regression of count value data via bayesian mixture of finite mixtures. In *Proc. of SIGKDD*. 3504–3512.
- [91] Zhongying Zhao, Hui Zhou, et al. 2021. Inductive Representation Learning via CNN for Partially-Unseen Attributed Networks. *IEEE Transactions on Net. Sci. and Eng.* 8, 1 (2021), 695–706.
- [92] Jialun Zheng, Jie Liu, Jiannong Cao, Xiao Wang, Hanchen Yang, Yankai Chen, and Philip S Yu. 2025. DP-DGAD: A generalist dynamic graph anomaly detector with dynamic prototypes. *arXiv preprint arXiv:2508.00664* (2025).
- [93] Y Zheng, H Wang, Z Wei, J Liu, and S Wang. 2022. Instant graph neural networks for dynamic graphs. In *Proc. of SIGKDD*. 2605–2615.
- [94] Y Zhong, H Vu, T Yang, and B Adhikari. 2024. Efficient and Effective Implicit Dynamic Graph Neural Network. In *Proc. of SIGKDD*. 4595–4606.
- [95] Yeqin Zhou and Heng Bao. 2025. Few-Shot Community Detection in Graphs via Strong Triadic Closure and Prompt Learning. *Mathematics* 13, 19 (2025), 3083.
- [96] Di Zhuang, J. Morris Chang, and Mingchen Li. 2021. DynaMo: Dynamic Community Detection by Incrementally Maximizing Modularity. *IEEE Trans. Knowl. Data Eng.* 33, 5 (2021), 1934–1945.

A Proof of Proposition 1

PROOF. We prove gradient isolation by showing that losses in orthogonal subspaces have zero cross-gradients. Let $\mathcal{F}(\mathbf{x}) = [\mathbf{W}_1\mathbf{x} \parallel \mathbf{W}_2\mathbf{x}]$ with $\mathbf{W}_1\mathbf{W}_2^\top = \mathbf{0}$ and $\mathcal{L} = \mathcal{L}_1(\mathbf{W}_1\mathbf{x}) + \mathcal{L}_2(\mathbf{W}_2\mathbf{x})$. Since $\mathcal{L}_1$ depends only on $\mathbf{z}_1 = \mathbf{W}_1\mathbf{x}$, we have

$$\nabla_{\mathbf{W}_2}\mathcal{L}_1 = \frac{\partial\mathcal{L}_1}{\partial\mathbf{z}_1} \cdot \frac{\partial(\mathbf{W}_1\mathbf{x})}{\partial\mathbf{W}_2} = \mathbf{0} \quad (21)$$

Similarly, $\nabla_{\mathbf{W}_1}\mathcal{L}_2 = \mathbf{0}$. Moreover, for any parameter θ affecting $\mathbf{x}$:

$$\nabla_{\theta}\mathcal{L}_1 = \frac{\partial\mathcal{L}_1}{\partial\mathbf{z}_1}\mathbf{W}_1\frac{\partial\mathbf{x}}{\partial\theta}, \quad \nabla_{\theta}\mathcal{L}_2 = \frac{\partial\mathcal{L}_2}{\partial\mathbf{z}_2}\mathbf{W}_2\frac{\partial\mathbf{x}}{\partial\theta} \quad (22)$$

The orthogonality $\mathbf{W}_1\mathbf{W}_2^\top = \mathbf{0}$ ensures gradient orthogonality:

$$\langle \nabla_{\theta}\mathcal{L}_1, \nabla_{\theta}\mathcal{L}_2 \rangle \propto \text{tr}(\mathbf{W}_1\mathbf{W}_2^\top) = 0 \quad (23)$$

Therefore, $\nabla_{\mathbf{W}_2}\mathcal{L}_1 = \mathbf{0}$ and $\nabla_{\mathbf{W}_1}\mathcal{L}_2 = \mathbf{0}$, enabling independent optimization without interference. $\square$

B Initialization of Node-Role Prototypes

As shown in Table 6, we initialize $\mathbf{f}^{\text{prior}}$ using binary assignments (1.0 for high intensity, 0.0 for low intensity) to ensure semantic distinctiveness. This sparse initialization serves two purposes:

- **Semantic separation:** Binary values force maximum separation in embedding space, preventing feature redundancy and accelerating convergence of the prototype alignment loss $\mathcal{L}_{\text{proto}}$.
- **Role disambiguation:** Contradictory feature assignments prevent semantic confusion—Newcomers have high volatility but zero centrality, distinguishing them from stable high-degree nodes.

This initialization strategy provides semantically grounded starting points essential for stable multi-level prototype learning. The specific features—centrality metrics (degree, betweenness, closeness), local structure indicators (clustering coefficient, intra community density), and temporal dynamics (volatility, stability)—are standard graph metrics widely used in network analysis.

C Community Evolution Patterns

We identify six fundamental evolution patterns based on role distribution changes $\Delta\gamma_k^t = \gamma_k^{t+1} - \gamma_k^t$:

- **Birth:** New community emerges at $t + 1$ with $|C_k^{t+1}| > \tau_{\min}$ and $|C_k^t| = 0$, typically initiated by Newcomers clustering together;

Table 6: The Node Role Prototype Prior Matrix (f^{prior}). The matrix encodes the ideal structural and temporal characteristics of the five predefined roles. A value of 1 indicates a required high intensity for the role, while 0 indicates a low intensity.

Prototype (p_i^n)	Centrality (f^c)			Local Structure (f^l)		Temporal (f^t)	
	Degree	Betweenness	Closeness	Clustering Coeff.	Intra-Density	Volatility	Stability
Community Leader (p_1^n)	1	0	1	1	1	0	1
Active Member (p_2^n)	1	0	0	1	1	0	1
Peripheral Member (p_3^n)	0	0	0	0	0	1	0
Bridge Node (p_4^n)	1	1	0	0	0	0	1
Newcomer/Evolver (p_5^n)	0	0	0	0	0	1	0

- **Death:** Community disappears with $|C_k^t| > 0$ and $|C_k^{t+1}| = 0$;
- **Growth:** $\Delta\gamma_{k,5} > 0.3$ (Newcomer influx) with $|\Delta\gamma_{k,1}| < 0.1$ (stable leadership), indicating healthy expansion phase;
- **Contraction:** $\Delta\gamma_{k,3} < -0.3$ (Wanderer exodus) with $\Delta\gamma_{k,1} > 0.1$ (leadership concentration), suggesting core group consolidation;
- **Split:** $\Delta\gamma_{k,4} < -0.5$ (Connector loss) and emergence of multiple components from C_k^t , reflecting internal fragmentation;
- **Merge:** $\Delta\gamma_{k,4} > 0.3$ (Connector increase) with community count reduction from t to $t+1$, showing successful integration;

These thresholds are empirically determined: $\tau_{\min} = 5$ (minimum community size), role change thresholds of 0.3 capture significant shifts while filtering noise, and the Connector threshold of 0.5 for splits ensures only major bridge losses trigger split detection. These patterns enable LLMs to reason about community dynamics through interpretable role-based changes.

D Semantic Description Generation

We design template function $\mathcal{T}(\cdot)$ to convert numerical graph metrics into hierarchical natural language descriptions. Box D presents the template specification that transforms role distributions, community compositions, and evolution patterns into interpretable narratives for LLM reasoning. The template ensures consistent generation of interpretable descriptions that preserve mathematical relationships while enabling natural language understanding for downstream LLM reasoning.

E Time Complexity Analysis

Time Complexity. The overall time complexity per snapshot is $O(L|E^t|d + |V^t|Kd_c + |V^t|d_r)$, where L is the number of GAT layers, d , d_c , d_r are the dimensions of embedding, cluster, and role subspaces respectively, and K is the number of communities. Specifically:

- GAT encoding: $O(L|E^t|d)$ for L layers with hidden dimension d .
- Modularity optimization: $O(|E^t| + |V^t|d_c)$ via our decomposition.
- Prototype learning: $O(|V^t|d_r)$ for role assignments.
- LLM reasoning: $O(|V^t|)$ API calls (can be batched).

Critically, our modularity reformulation reduces complexity from $O(|V^t|^2)$ to $O(|E^t|)$, enabling scaling to graphs with millions of nodes. For sparse graphs where $|E^t| = O(|V^t|)$, the total complexity becomes linear in the number of nodes.

Space Complexity. The space requirement is $O(|V^t|(d + d_c + d_r) + Kd_c)$, consisting of:

- Node embeddings: $O(|V^t|d)$ for temporal states.
- Orthogonal projections: $O(|V^t|(d_c + d_r))$.
- Cluster centroids: $O(Kd_c)$ where $K \ll |V^t|$.

The decomposition ensures $d_c + d_r \leq d$, preventing memory overhead. For web-scale deployment, embeddings can be computed in mini-batches, requiring $O(bd)$ memory where b is batch size.

Scalability. Our framework scales to million-node graphs through: (i) linear-time modularity optimization avoiding dense matrix operations, (ii) sparse attention in GAT processing only existing edges, and (iii) batched LLM inference with caching for repeated patterns. Empirically, we process graphs with 1M+ nodes in under 10 minutes on a single GPU (see Section 5).

Box 1. LLM Reasoning Process for Community Correction

Initial Structural Clustering: $v_{42} \rightarrow C_1$ (ML Community, confidence: 0.75)
Semantic Description Input:
Node: Node v_{42} exhibits 45% Connector and 40% Contributor characteristics, with 28 connections across multiple research areas.
Community: C_1 : 60% Contributors, 30% Leaders, 5% Connectors (ML community).
 C_3 : 20% Connectors, 35% Contributors, 15% Newcomers (Bio-AI community).
Evolution: " C_1 stable with internal focus. C_3 rapidly growing with 25% Newcomer influx, seeking bridge nodes."
LLM Chain-of-Thought Reasoning:
Step 1: Connector role (45%) indicates bridging function, not peripheral member.
Step 2: C_1 has role mismatch (needs Contributors, has excess Leaders).
Step 3: C_3 has role alignment (needs Connectors for interdisciplinary growth).
Step 4: Evolution fit - growing C_3 requires bridges, stable C_1 does not.
Step 5: Reassign to C_3 despite fewer edges (functional > structural).
Corrected Assignment: Node $v_{42} \rightarrow$ Community C_3 (confidence: 0.82)
Validation: Modularity: 0.68 $\rightarrow$ 0.71 | Role Coherence: Improved.
Ground truth: v_{42} confirmed as Bio-AI researcher

F Case Study

As shown in Box 1, DyG-MF [28] classifies v_{42} into C_1 (ML community) with 0.75 confidence, treating it as a wanderer ML researcher, despite 8/10 recent papers being Bio-AI collaborations. DyG-RoLLM correctly identifies v_{42} 's Connector role (45%) and reassigns them as a Bio-AI bridge researcher rather than a peripheral ML member. This role-aware reasoning improves modularity (0.68 $\rightarrow$ 0.71) while capturing the nuanced function that topology-based methods miss when they see only the 18/28 ML edge majority.